

Présentation de Spark

Veille technologique de Abderrazak, Frédéric et Samir

Spark c'est quoi ?

- Moteur de traitement distribué pour le Big Data.
- Permet de traiter de très grandes quantités de données sur plusieurs machines.

Spark après Hadoop et MapReduce

Apache Spark

Traitement en mémoire (RAM)

Plus rapide pour de nombreux traitements

Plus polyvalent : batch, streaming, SQL, machine learning, graphes

API plus modernes et simples

Hadoop MapReduce

Écrit les données sur le disque entre les étapes

Pus lent à cause des lectures/écritures disque

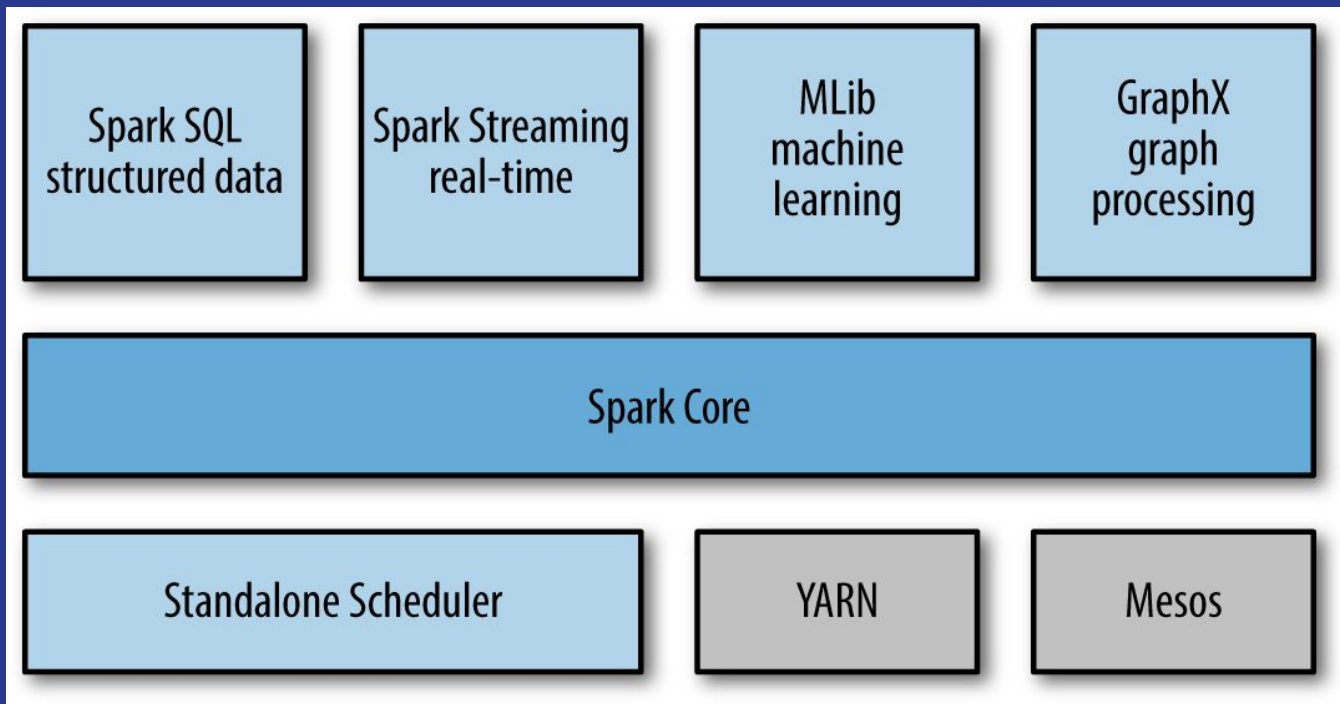
Principalement orienté traitement batch

Modèle de programmation plus rigide

Composants de Spark

- **Spark Core**
- **Spark SQL**
- **Spark Streaming / Structured Streaming**
- **MLlib**
- **GraphX**

Structure générale de Spark



Fonctionnement de Spark Core

- Création du contexte Spark
- Découpage du travail et Distribution sur le cluster
- Gestion des RDD et des pannes

Fonctionnement de Spark SQL

- Lecture des données structurées
- Création de DataFrames
- Exécution de requêtes SQL select, filter, groupBy et join...

Modes de fonctionnement

- Apache Spark fonctionne en exécutant des tâches de traitement sur des grands ensembles de données à l'aide de machines de Clusters
- Apache Spark permet de réaliser des traitements par lot (batch processing), ou à la volée (stream processing) par l'outil Spark Streaming qui traite les flux de données en direct

Fonctionnement en streaming

- les données arrivent progressivement (événements, logs, transactions, messages, ...)
- l'application reste toujours active
- les traitements sont effectués continuellement ou plus souvent par micro-batches (par ex. données regroupées chaque seconde)
- adapté au monitoring, détection d'anomalies, tableaux de bord temps réel
- possibilité de gérer des fenêtres temporelles (nombre de transactions sur les 5 dernières minutes)
- sources typiques : Kafka, événements, sockets

Fonctionnement en batch

- traite des données déjà disponibles, généralement en une seule exécution
- les données sont traitées par lots
- les traitements sont exécutés ponctuellement (chaque nuit)
- adapté aux traitements historiques, ETL, reporting, calculs massifs
- le débit est privilégié à la faible latence
- sources typiques : fichiers, tables

Résilience de Spark

Tolérance aux pannes, notamment grâce aux RDD (Resilient Distributed Datasets), qui permettent de reconstruire des données en cas de problème.

Si une partition est perdue, Spark peut la recalculer à partir des données sources et des transformations précédentes, sans devoir tout recommencer.

Le bon fonctionnement des traitements est contrôlé par des heartbeats des exécutants, ou par la surveillance du cluster manager (Yarn).

Scalabilité de Spark

- scalabilité horizontale : traitements sur plusieurs machines du cluster
- parallélisation : répartition des données à traiter par partitions, distributions des calculs par noeuds
- gestion des noeuds défaillants par le mécanisme de lineage
- limites : communications entre noeuds, opérations de shuffle (réorganisation des données entre partitions pour regrouper celles ayant la même clé dans un même noeud)

Cas d'usage

Les cas d'usage de Spark sont surtout liés au traitement de gros volumes de données, en batch ou en temps réel.

- ETL / ELT
- Traitement Big Data
- Analyse SQL distribuée avec Spark SQL
- Streaming temps réel avec Structured Streaming
- Machine Learning avec MLlib
- Traitement de fichiers volumineux
- Data Lake / Lakehouse
- Analyse de logs

Cas d'usage

Cas d'usage Spark — e-commerce

Transformer des données de ventes en indicateurs exploitables pour le pilotage métier.

Objectif

Centraliser les commandes, clients et produits pour calculer automatiquement les KPI commerciaux.

Traitements Spark

- Nettoyage des données
- Jointures commandes / clients / produits
- Agrégations : CA, panier moyen, top produits

Résultat

Des fichiers Parquet prêts pour un Data Warehouse ou un tableau de bord Power BI.



Exemple : CSV / base de données → Spark → Parquet → BI

Bilan — Apache Spark

Un moteur distribué pour transformer et analyser de grands volumes de données.

À retenir

Spark est pertinent quand les données deviennent trop lourdes pour Pandas ou un SQL local.

**Données
massives**

**Calcul
distribué**

**Décisions
fiables**

Ce que Spark apporte

- Traitements parallélisés sur plusieurs machines
- SQL, ETL, streaming et ML dans un même écosystème
- Lecture efficace des formats Parquet / Delta / Iceberg

À utiliser quand...

Les volumes sont importants, les calculs sont longs ou les données arrivent en continu.

À éviter quand...

Le dataset tient facilement en mémoire : Pandas, DuckDB ou SQL classique suffisent souvent.

Conclusion : Spark est un outil puissant pour industrialiser des pipelines Big Data, mais il doit être choisi quand le besoin justifie sa complexité.

Spark vs les technologies concurrentes

Comparaison rapide des principales solutions de traitement de données

Technologie	Usage principal	Points forts	Limites / quand l'éviter
 Apache Spark	Batch, SQL, streaming, ML	✓ Polyvalent, distribué, très utilisé en data engineering	✗ Lourd pour de petits volumes
 Apache Flink	Streaming temps réel	✓ Très performant en streaming, faible latence	✗ Moins orienté analyse batch généraliste
 Apache Kafka	Transport d'événements	✓ Excellent pour ingérer et diffuser des flux	✗ Ce n'est pas un moteur de transformation analytique complet
 Hadoop MapReduce	Traitement batch historique	✓ Robuste sur gros volumes	✗ Plus lent et moins flexible que Spark
 DuckDB	Analyse locale	✓ Très rapide sur une seule machine	✗ Pas conçu pour le calcul distribué massif
 Polars	DataFrame rapide	✓ Très rapide, moderne, efficace	✗ Moins adapté aux clusters Big Data
 Pandas	Analyse locale / exploration	✓ Simple, très connu, idéal pour petites données	✗ Limité en mémoire et en volumétrie



À retenir

Spark est souvent le meilleur compromis quand on veut traiter de gros volumes avec des besoins variés : ETL, SQL, batch et parfois streaming.